Support et déroulé de la soutenance - MiniShop (15 minutes)

Document n° 4 du rendu (barème iv : soutenance + questions, 10 points). Le plan ci-dessous est **chronométré** et répartit la parole entre les 3 personnes ; chaque minute est reliée à une section du cahier des charges, de sorte qu'aucune affirmation ne soit hors document.

1. Matériel à préparer

• support `docs/03-support-soutenance.md` exporté en PDF (12 diapositives, une par minute) ;

• poste prêt : `php -S 127.0.0.1:8000 -t public` lancé, base **rejouée à froid** (`./scripts/load_db.sh`),

deux navigateurs ouverts (front-office en navigation privée côté client, back-office à côté) ;

• terminal ouvert sur `mysql minishop` avec les 4 requêtes de preuve §12.3.5 déjà collées ;

• capture figée de la sortie de `./scripts/run_sql_tests.sh` (28/28) en secours si le réseau/la machine flanche ;

• liens GitLab prêts (dépôt, MR, `git shortlog -sne`, README) ;

• **filet de sécurité** : 3 minutes de démo enregistrées (vidéo locale) au cas où rien ne se lance.

2. Déroulé minute par minute

min	Qui	Contenu	Preuve affichée	Réf. CDC
0-1	A	Contexte de l'entreprise, problème du tableur partagé, ce que MiniShop doit changer ; acteurs	1 slide (état actuel + 3 acteurs)	§1.1, §3
1-2	A	Périmètre strict (in/out), objectifs mesurables, règle anti-dérive	table §13.1 / §2.3	§2, §11.2
2-4	B	**Modélisation** : MCD (7 entités), 3 cardinalités défendues, MLD, normalisation (3FN, 2 dénormalisations protégées)	`mcd_minishop.png`, tableau §12.1.3	§12.1-§12.3
4-6	B	**SQL** : script rejoué en direct (`load_db.sh` : 8 tables, 3 vues, 1 fonction + 17 procédures, 16 triggers) + les 4 requêtes de preuve qui renvoient 0 ligne	terminal	§12.4, §12.3.5
6-8	B	**Procédures stockées** : `sp_add_order_line` ligne à ligne (verrou, snapshot, décrément, montant) ; réponse à la question pédagogique (centralisation, sécurité, transactions, réutilisabilité, contrôle d'accès)	code + §12.5.1	§12.5
8-9	B	**Triggers** : les 2 imposés (stock, historique) + les 4 de protection ; montrer qu'un `UPDATE` manuel est refusé	terminal : `UPDATE produit SET stock = -5` -> `RB03_STOCK_NE_PEUT_PAS_ETRE_NEGATI	§12.6, T-03

9-10	C	**Tests** : `run_sql_tests.sh` rejoué -> 28/28, en insistant sur les 15 tests qui contournent l'application + le registre d'anomalies (A-01 à A-08, dont A-08 trouvée **en mesurant** la volumétrie réelle)	terminal + §7 du doc de tests	§15, doc tests
10-1	C	**Démo front** : recherche (filtres dans l'URL), fiche produit, panier avec plafond de stock, commande validée -> numéro, stock qui baisse	navigateur	§4.2-§4.4
12-13	C	**Sécurité en direct** : (1) payload SQLi sans effet, (2) nom de produit `<script>` affiché en texte, (3) `?id=` d'un autre client -> 404, (4) prix falsifié ignoré (snapshot)	navigateur + requêtes	§8
13-1	A	**Back-office** : création d'un produit (TTC calculé en base), ajustement de stock refusé en négatif, changement de statut + historique, et **une transition interdite refusée**	navigateur	§4.5, F-22/F-26/F-29
14-15	A	Organisation : MVC en 3 couches (1 slide), GitLab (3 auteurs, MR par lot, CI qui casse sur une concaténation SQL), livrables et difficultés, ce qui est volontairement hors périmètre	`git shortlog`, `.gitlab-ci.yml`	§9, §14, §11.2

Contraintes de tenue du temps : pas de lecture de slide, chaque personne/terms tenus en <= 2 min sans dépassement, et la démonstration SQL se fait dans un terminal pré-rempli (pas de saisie longue). Deux répétitions chronométrées sont planifiées (S11, S12) - les dérives typiques sont la modélisation qui mange la démo, et la démo qui mange les questions.

3. Questions anticipées et réponses courtes (à connaître par cœur)

Question probable	Réponse (10 à 30 s)	Réf.
Pourquoi une procédure stockée plutôt que du PHP pour la commande ?	Atomicité et verrous côté serveur, logique unique pour 4 appelants, privilèges réduits sur les tables, 1 aller-retour au lieu de 2N+2 ; le coût est la portabilité et le debuggage - nous l'assumons pour les écritures seulement	§12.5.1
Pourquoi garder `prix_ttc` et `montant_total` si c'est calculable ?	Deux dénormalisations **tracées et verrouillées** par trigger : elles portent la valeur contractuelle du moment de l'achat (facturation) et évitent une agrégation à chaque affichage ; sans le trigger, la redondance serait une anomalie	§12.3.2, §12.3.4
Pourquoi `total_ligne` et pas `prix × quantité` à la demande ?	Pour la même raison : le prix payé ne doit plus jamais bouger ; le trigger le recalcule à l'insertion et l'immutabilité l'interdit après	RB-06, RB-09, T-11

Pourquoi deux tables CLIENT et ADMINISTRATEUR et pas une table UTILISATEUR avec un rôle ?	Données et cycle de vie disjoints, aucune écriture possible sur la table admin depuis le front (SEC-10), contrôle d'accès plus simple à prouver ; le surcoût est nul à notre échelle	V4 du §12.8
Pourquoi le `BROUILLON` existe-t-il, on ne peut pas créer la commande directement en `EN_PREPARATION` ?	Sinon `RB-04` (au moins une ligne) est violée à la création ; le brouillon est l'état « panier convertible », jamais exposé à l'admin (`COMMANDE_NON_TRAITABLE`)	V3 du §12.8, T-16
Comment est garantie l'absence de survente ?	Verrou `SELECT ... FOR UPDATE` sur la ligne de stock + contrôle dans la procédure **et** dans le trigger `BEFORE INSERT` ; le test de charge : 20 demandes sur 12 exemplaires -> 12 commandes, 8 refus, stock 0	§2.3, C-01
Que se passe-t-il si un développeur oublie le contrôle d'appartenance dans un contrôleur ?	La procédure refuse (`ACCES_NON_AUTORISE`) : le test `T-18` appelle `sp_confirm_order` avec le mauvais client et vérifie le rejet - c'est notre preuve de défense en profondeur	SEC-08, T-18
Pourquoi un trigger et pas la procédure pour écrire l'historique ?	Sinon un `UPDATE` manuel (import, reprise, correction) ne serait pas tracé ; et le test `T-22` vérifie aussi l'**absence de doublon**, ce qui prouve que la procédure n'écrit pas	A-02, T-22
Comment prouvez-vous que vous n'avez pas d'injection SQL ?	3 verrous : requêtes préparées réelles (`EMULATE_PREPARES=false`), tout SQL dynamique déplacé dans la procédure, contrôle `grep` bloquant en CI ; plus le test d'attaque S-01 avec 12 payloads	§8.1, §15.3
Et si on passe par phpMyAdmin pour modifier le stock à -5 ?	Refusé par le CHECK et par le trigger (`RB03_...`) : c'est le test `T-02`/`T-03`, exactement le scénario demandé	§6.2
Pourquoi le mot de passe est-il vérifié en PHP et pas en base ?	Seul `password_verify()` sait comparer bcrypt en temps constant ; la base ne renvoie donc que le hash via `sp_get_credentials`, la comparaison reste dans l'application	§8.4, séquence 1
Quelle règle vous a posé le plus de difficulté, et pourquoi ?	`RB-06` + immuabilité : nous avons d'abord laissé le trigger de snapshot « réparer » silencieusement les UPDATE au lieu de les refuser ; l'ordre de création des triggers est déterministe dans MySQL, nous l'avons corrigé et figé par le test `T-11`	A-01
Que feriez-vous en poursuite (hors périmètre) ?	Panier persistant (variante V1 prête), paiement réel avec journal de transactions, politique de prix par remise (une table `REMISE` + snapshot), export comptable, et tests Playwright bout en bout	§12.8, §11.2
Où est la contribution de chacun dans GitLab ?	3 auteurs, une MR par lot du §13.3, commits par zone (front/back/base pour chacun) ; la CI échoue sur une concaténation SQL ou un test de base non conforme	§14

Faut-il relancer la CI après un merge ?	Non : la MR est déjà testée **sur son commit de fusion** (`refs/pull/<n>/merge`, *merge result* GitLab), donc si la PR est verte, le run post-merge rejoue les mêmes 6 jobs sur le même arbre et ne peut rien révéler de plus. On ne relance que si `main` a avancé, si le run a été annulé, ou après une panne d'infrastructure	§14.2
Une MR peut-elle être mergée avec un pipeline rouge ?	Non : `php-lint`, `security-scan`, `base-tests`, `unit` et `docs` sont bloquants (`style` est le seul en `allow_failure`) ; c'est ce qui rend la pénalité visible pendant le développement et non au rendu	§14.2

3.1 Les trois décisions « hors sujet » à défendre en 30 secondes chacune

Décision	Phrase à dire	Si le jury insiste
Les frais de port sont dans le périmètre	« Ce n'est pas une fonctionnalité de confort : l'article **L221-5 4° du code de la consommation** et l'article **5 §1 e) de la directive 2011/83/UE** imposent d'afficher le prix total **y compris les frais de livraison avant la validation** du panier. Sans cela le professionnel ne peut tout simplement pas les réclamer. »	Montrer `RB-20`, `sp_compute_shipping`, `T-28` (port calculé avant validation, **figé** à la validation, **non saisissable**) et la table `parametre` : la règle 4,90 EUR / franchise 80 EUR se change sans toucher au code. Préciser ce qui reste **exclu** : transporteur, tarif par pays, suivi de colis.
Ventes interdites à stock nul	« `RB-04`/`RB-18` : on ne vend pas ce qu'on n'a pas. Le stock est réservé ligne par ligne sous verrou `FOR UPDATE`, et un `CHECK (stock >= 0)` - un trigger bloquent même une écriture SQL directe. »	Exécution en direct : `UPDATE produit SET stock = -1` -> `RB03_STOCK_NE_PEUT_PAS_ETRE_NEGATIF` (`T-02`).
Deux rôles d'administrateur, pas de MFA	« L'énoncé dit simplement << rôle administrateur >>. On a appliqué le moindre privilège - **OWASP ASVS V4.1.3** et le guide **ANSSI** R27/R29 - en séparant `SUPER` (comptes d'équipe) de `GESTIONNAIRE` (catalogue et commandes). La MFA, exigée par ASVS V4.3.1, est **explicitement hors périmètre et écrite comme un écart** au §8.10 du cahier des charges, avec ce qu'il faudrait pour la livrer. »	Montrer `EF-ADM-10` (garde `SUPER`), `F-32` (un `GESTIONNAIRE` reçoit 403 sur la gestion d'équipe) et le §8.10 (tableau des écarts).
Volumétrie chiffrée, pas supposée	« Le client visé fait **200 produits et 1 000 commandes par an** : on a donc généré ce volume avec `scripts/gen_volumes.sh` - **par appels aux procédures**, pas par des `INSERT` bruts - pour que `ENF-01` (< 500 ms p95) soit mesuré sur des données réalistes et cohérentes avec les règles. »	Montrer le bilan du générateur : `ttc_incoherents = 0`, `montants_incoherents = 0` sur le jeu généré : les règles tiennent aussi à 1 000 commandes.

4. Check-list de la dernière demi-heure avant l'entrée en salle

- [ ] `./scripts/load_db.sh` rejoué à froid, compteurs corrects (8 tables, 3 vues, 1 fonction, 17 procédures, 16 triggers, 12 produits, 4 commandes de démo)
- [ ] `./scripts/run_sql_tests.sh` -> **27 / 27**
- [ ] 1 page de chaque zone ouverte et fonctionnelle (catalogue, fiche, panier, commande, admin produits, admin commandes)
- [ ] les 4 requêtes de preuve §12.3.5 collées dans le terminal, prêtes à être exécutées
- [ ] les 3 requêtes « attaques » prêtes : `UPDATE produit SET stock = -5 ...`, `CALL sp_confirm_order(...)`, `SELECT COUNT(*) FROM client`
- [ ] `git shortlog -sne main` exécuté et visible (3 auteurs)
- [ ] diaporama exporté en PDF, projeté sans connexion
- [ ] chrono réglé sur 15 min, une personne désignée « gardien du temps »